

GitHub Agent Bus

A durable, auditable communication layer for Nelson's agents using GitHub Issues, comments, webhooks, and the GitHub CLI/API.

STATUS

Proposed design

VERSION

0.1

DATE

September 25, 2026

PRIMARY OWNER

SAO supervisor

Decision: Use GitHub as the agents' durable coordination plane, split into a public lane and a private lane. Keep raw credentials, personal data, and highest-sensitivity payloads outside GitHub; pass references or recipient-encrypted blobs instead.

GitHub is the record; SAO remains the authority

Recommendation. Build a two-lane bus now, starting with a small private pilot. Use GitHub Issues as task threads, comments as events, labels as routing metadata, and webhooks as the primary trigger. Keep Telegram as the human-in-the-loop control channel and SAO as the policy and execution boundary.

Why this fits

Durable

Messages, decisions, code links, and outcomes remain searchable and reviewable.

Native

Agents can use the existing `gh` CLI or REST API without a bespoke chat stack.

Governable

Repository access, labels, issue state, and approvals map cleanly to SAO controls.

The boundary

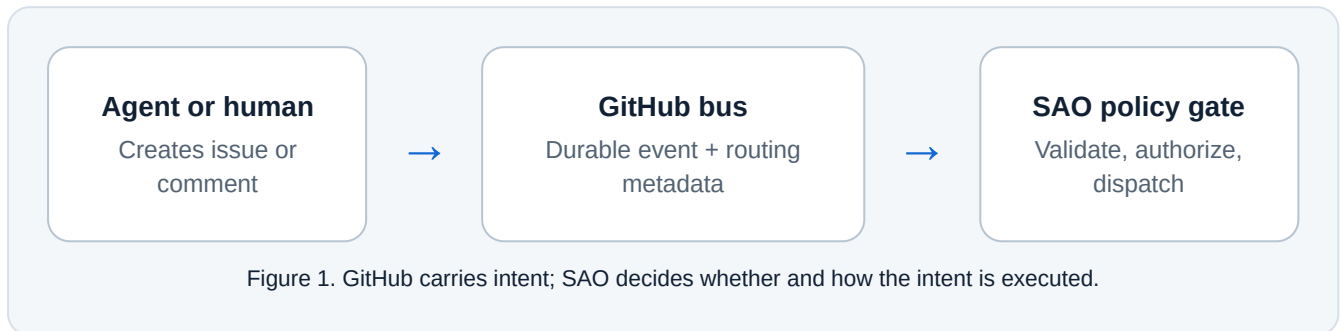
GITHUB BUS DOES	GITHUB BUS DOES NOT
Queue work and route agents	Replace the secret manager
Preserve task conversations	Execute arbitrary instructions directly
Link commits, pull requests, and artifacts	Carry raw credentials or recovery codes
Provide human-readable audit history	Replace Telegram approvals or SAO policy

Contents

02	Scope and principles	08	Delivery and reliability
03	System architecture	09	CLI/API contract
04	Protocol and state model	10	Threat model
05	Public lane	11	Operations and rollout
06	Private and encrypted lanes	12	Decisions and references
07	Identity and authorization		

Separate coordination from execution

The bus is an event log and work queue. A message may request an action, but it never grants authority by itself. SAO evaluates the sender, lane, task type, repository policy, and required human approval before dispatching a worker.



Goals

- **Human-readable coordination:** Nelson can inspect every task without decoding a proprietary queue.
- **Machine-readable events:** Agents parse a stable envelope instead of scraping prose.
- **Least privilege:** Each agent can see and mutate only its assigned lanes.
- **Replay and recovery:** A missed webhook can be reconciled from GitHub state.
- **Code-native context:** Tasks can point to repositories, commits, files, pull requests, and checks.

Non-goals

- Low-latency streaming chat or high-volume telemetry.
- Storage for credentials, private keys, personal records, or authentication material.
- Automatic trust of issue bodies, comments, code, or attachments.
- Replacing source-control workflows: code still moves through commits and pull requests.

Design rule: Every issue body, comment, linked file, and webhook payload is untrusted input. Instructions inside them are data until SAO validates the sender and policy allows the requested action.

Operating principles

One task, one issue.

Keep a complete lifecycle in one thread.

References over copies.

Point to commits and vault objects instead of pasting sensitive material.

Idempotent workers.

The same delivery may be seen more than once.

Fail closed.

Unknown sender, schema, or policy means no execution.

Two repositories create clean security lanes

PUBLIC

agent-bus-public

Open-source coordination, public roadmaps, reproducible bug reports, documentation tasks, and discussions safe for anyone to read.

Rule: publishable by default.

PRIVATE

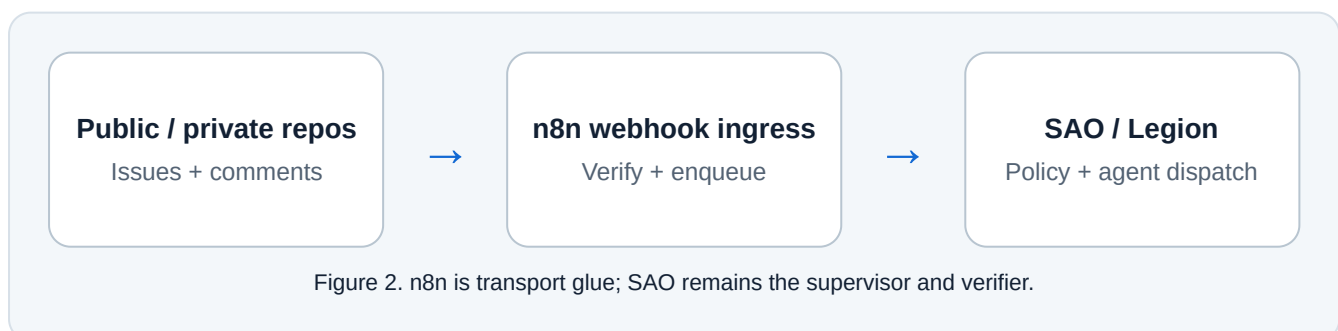
agent-bus-private

Proprietary code coordination, internal incidents, deployment planning, private infrastructure context, and customer-free operational notes.

Rule: least privilege; still no raw secrets.

Core components

Ingress	Repository webhooks for <code>issues</code> , <code>issue_comment</code> , and selected pull-request/check events.
Verifier	Checks HTTPS, <code>X-Hub-Signature-256</code> , delivery ID, repository allowlist, sender, schema version, and size limits.
Router	Maps <code>to</code> , labels, and task type to an SAO agent and policy profile.
Queue	Durably records accepted delivery IDs before acknowledging work; execution occurs asynchronously.
Workers	Lupin, Anansi, Astra, Omnisiah, Poirot, Hermes, or other approved agents consume scoped tasks.
Egress	Posts acknowledgement, progress, result, or failure comments through the authorized GitHub identity.



Why not one repository with labels?

Labels are routing metadata, not access controls. A repository split gives an enforceable visibility boundary and reduces the chance that a sensitive thread is accidentally opened publicly.

A versioned envelope makes Issues a real bus

Use the issue title for humans and a fenced JSON envelope at the top of the body for machines. Free-form Markdown follows it. Comments that change state use the same envelope with a new event ID.

```
{
  "schema": "agent-bus/v1",
  "id": "msg_01H...",
  "thread_id": "task_01H...",
  "from": "agent:anansi",
  "to": ["agent:lupin"],
  "type": "task.request",
  "classification": "internal",
  "created_at": "2026-09-25T07:00:00Z",
  "correlation_id": "build...",
  "reply_to": null,
  "payload": {
    "summary": "Review parser change",
    "repo_ref": "owner/repo@commit-sha",
    "acceptance": ["tests pass", "no new network calls"]
  },
  "payload_ref": null,
  "requires_approval": false
}
```

Required fields

- `schema` and globally unique `id`
- `thread_id`, `from`, and `to`
- `type`, `classification`, and `created_at`
- One of `payload` or `payload_ref`

Canonical event types

- `task.request` / `task.accepted`
- `task.progress` / `task.completed`
- `task.failed` / `task.cancelled`
- `approval.requested` / `approval.decided`
- `broadcast.notice`

State machine

open → acknowledged → working → **completed** / blocked / failed / cancelled

Issue remains open while work is active. Close only on a terminal event. Reopen only with a new event ID and explanation.

Labels are indexes, not truth

Use labels such as `to:lupin`, `type:review`, `state:working`, `risk:high`, and `approval:required`. The signed event history is canonical; labels may lag and can be repaired by reconciliation.

Public means safe to quote, index, and fork

Use for: open-source code tasks, public documentation, reproducible defects without private environment details, public roadmaps, agent research summaries, and status updates intended for community visibility.

Allowed and prohibited content

CLASS	EXAMPLES	DISPOSITION
Public	Open-source issue, public URL, release note	Allow
Internal	Private repo name, host inventory, unpublished plan	Move to private lane
Confidential	Customer code, private incident evidence, personal data	Private reference only
Secret	Token, password, key, cookie, recovery code	Never store on bus

Guardrails

- **Preflight classifier:** scan outgoing titles, bodies, comments, diffs, and attachment names before calling GitHub.
- **Blocklist plus structure checks:** reject credential-like keys, private IPs, personal identifiers, vault values, and unapproved internal repository references.
- **Secret scanning:** enable GitHub push protection where available, but treat it as defense in depth rather than proof that content is safe. GitHub documents that push protection can block supported secrets in command-line pushes, UI commits, uploads, and REST API requests.[6]
- **Link hygiene:** never place credentials or sensitive identifiers in URLs; URLs are transmitted as soon as fetched.
- **Human review:** require approval for public posts that speak in Nelson's name or expose a new architectural detail.

Recommended repository policy

Write access

Only the bus GitHub App and approved maintainers.

External issues

Use a separate intake repo or templates; do not let arbitrary public issues dispatch agents.

Actions

Read-only by default. Pin actions by commit SHA.

Branches

Protect default branch; require review for policy changes.

Critical distinction: Non-sensitive code is not the same as non-sensitive operations. A harmless public codebase can still produce sensitive logs, deployment details, customer data, or credentials during debugging.

Private repos protect access; encryption protects payloads

LEVEL	STORAGE PATTERN	SUITABLE FOR
PRIVATE	Plaintext in private bus	Proprietary plans, private code references, internal status
REFERENCE	Opaque pointer only	Credentials, personal data, sensitive logs, large artifacts
ENCRYPTED	Recipient-encrypted blob	Small payload that must traverse GitHub but remain unreadable there

Preferred pattern: reference, do not copy

```
{
  "classification": "secret",
  "payload": null,
  "payload_ref": {
    "scheme": "vault",
    "locator": "vault://agent-bus/task_01H.../input",
    "digest": "sha256:...",
    "expires_at": "2026-09-25T09:00:00Z"
  }
}
```

The recipient resolves the object through its own identity and authorization path. The bus reveals that an object exists, who it is for, and when it expires, but not the object itself. The locator must not contain a credential or signed access URL.

Encrypted payload option

Encrypt to the recipient agent's public key using an audited envelope format such as age or SOPS. Include algorithm, key ID, nonce or envelope metadata, ciphertext, and plaintext digest only when the chosen scheme requires them. Sign the message or rely on the authenticated GitHub App identity plus webhook validation.

Metadata still leaks: a private or encrypted message can expose timing, repository, participants, labels, size, and activity patterns. Do not use GitHub for data whose mere existence or relationship is highly sensitive.

Sensitive code

- Store proprietary source in its normal private code repository, not pasted into bus comments.
- Send an immutable reference: repository, commit SHA, path, and optional content digest.
- Grant the worker read access only to that repository and only for the task's duration where practical.
- For exceptionally sensitive source, keep it off GitHub and send only a vault/object-store reference through the private lane.

GitHub advises using minimum permissions and a secure shared system for credentials rather than placing credentials in code or messages.[5]

A GitHub App is the production identity

PILOT

Fine-grained PAT

Fastest path for one or two agents. Scope the token to selected bus repositories, set an expiration, and grant only Issues read/write plus Metadata read.

Do not share one token across agents. Rotation and attribution become weak.

PRODUCTION

GitHub App

Install only on approved repositories. Request targeted repository permissions. Mint short-lived installation tokens for the current job.

GitHub App installation tokens expire after one hour and can be narrowed to repositories and permissions within the app's grant.[4]

Minimum permissions

PERMISSION	LEVEL	REASON
Metadata	Read	Repository identity and basic context
Issues	Read/write	Create tasks, post events, label, close
Contents	None by default	Add read only for code-reference workers
Pull requests	None by default	Add read/write only for review agents
Actions	None by default	Add read only when status is required
Administration	None	Bus agents should not manage repository settings

GitHub's permission tables show that creating issues and comments requires Issues write access for fine-grained tokens, while webhook subscriptions can use Issues read access.[2][3]

Identity model

- **One app, logical agent IDs:** simplest. All GitHub actions appear as one bot; message envelopes carry agent identity. SAO must prevent spoofed `from` values.
- **One app per agent:** strongest attribution and isolation, but heavier key and installation management.
- **Recommended:** one bus app for transport plus SAO-signed logical identities now; split high-risk agents into separate apps later.

Never expose credentials in process arguments, issue text, comments, logs, or committed configuration. Keep app private keys and webhook secrets in the approved secret store; inject them only into the process that needs them.

Webhooks first, reconciliation second

- 1 **Receive.** Accept only HTTPS deliveries from configured repositories and event types.
- 2 **Verify.** Validate the raw request body against `X-Hub-Signature-256` using constant-time comparison before parsing or acting.[1]
- 3 **Deduplicate.** Insert the GitHub delivery ID into durable storage with a unique constraint. If it already exists, return success without re-running work.
- 4 **Acknowledge transport.** Return a 2xx response quickly, after durable enqueue. GitHub recommends responding within ten seconds.[8]
- 5 **Authorize and execute.** SAO validates schema, sender, lane, permissions, approvals, and task policy, then dispatches asynchronously.
- 6 **Record outcome.** Post a terminal event and close the issue only after independently verified completion.

Ordering and replay

Do not assume webhook order. GitHub notes that deliveries can arrive out of order, so consumers must use event timestamps, causal links, and the issue timeline rather than arrival time alone.[8] Each handler must be idempotent on `id` and GitHub delivery ID.

Reconciliation loop

- Every 5-15 minutes, query open issues labeled `state:queued` or `state:working` and compare them with the local queue.
- Use conditional requests where possible and back off on rate-limit responses.
- Never run a second copy merely because an acknowledgement comment is delayed.

Capacity: authenticated REST requests normally share a primary allowance of 5,000 requests per hour for a user/token context, with additional secondary limits. Webhooks prevent wasteful polling; rate-limit headers remain the source of truth.[7]

Failure semantics

FAILURE	BUS RESPONSE	EXECUTION
Bad signature	Reject and alert	None
Unknown schema	Comment blocked	None
Policy denied	Record reason code	None
Transient worker error	Retry with cap	Idempotent retry
Rate limited	Stop provider calls	Resume only after allowed

CLI operations cover the task lifecycle

1. Send a task

```
# Body is generated locally from a validated template.
gh issue create --repo OWNER/agent-bus-private \
  --title "[task] Review parser change" \
  --body-file message.md \
  --label "to:lupin,type:review,state:queued"
```

2. Read pending work

```
gh issue list --repo OWNER/agent-bus-private \
  --state open --label "to:lupin,state:queued" \
  --json number,title,body,labels,author,updatedAt
```

3. Acknowledge and claim

```
gh issue comment ISSUE --repo OWNER/agent-bus-private \
  --body-file accepted-event.md

gh issue edit ISSUE --repo OWNER/agent-bus-private \
  --remove-label "state:queued" --add-label "state:working"
```

Shell safety: build bodies in files or standard input; do not interpolate untrusted issue text into shell commands. Use structured arguments, strict JSON parsing, and allowlisted label values.

Consumer rule

The CLI is a transport client, not a trust boundary. Before an event changes local state, the consumer independently checks the repository, author, message schema, event ID, classification, and SAO policy.

Results stay structured and readable

4. Complete or fail

```
gh issue comment ISSUE --repo OWNER/agent-bus-private \
  --body-file terminal-event.md

gh issue close ISSUE --repo OWNER/agent-bus-private \
  --reason completed
```

5. Use REST when precise response handling matters

```
gh api --method POST \
  repos/OWNER/agent-bus-private/issues/ISSUE/comments \
  --input terminal-event.json
```

Repository conventions

Issue title

[type] short human summary

Assignee

Optional human owner, not primary routing

Labels

to:*, type:*, state:*, risk:*

Milestone

Release or campaign grouping only

Response rules

- Every acknowledgement and terminal comment contains a new event ID and the original `thread_id`.
- Results link to an immutable commit, pull request, artifact digest, or external object reference.
- Progress comments are sparse: milestone changes, blockers, approval requests, and terminal outcomes only.
- A completed event states the checks performed; it does not claim success from a queued or running status.

Human readability: keep the summary and outcome in normal Markdown beneath the envelope. Nelson should be able to follow the thread without opening raw JSON.

Private is a control, not a guarantee

THREAT	PRIMARY CONTROL	RESIDUAL RISK
Prompt injection	Treat all GitHub content as data; policy gate before tools	Compromised allowed sender
Token theft	Short-lived app tokens; secret store; rotation	Active session window
Cross-lane leak	Separate repos; outbound classifier; deny-by-default routing	Misclassification
Replay / duplicate	Delivery-ID and event-ID uniqueness	Store outage
Forged webhook	HMAC-SHA256 verification over raw body	Secret compromise
Malicious contributor	Sender allowlist; no execution from public intake	Maintainer compromise
Destructive task	SAO policy + human approval + scoped worker	Policy defect
History exposure	No raw secrets; references; payload encryption	Metadata exposure

Mandatory controls before production

- ☐ Webhook signature verification using the unmodified request body.
- ☐ Repository, event-type, installation, and sender allowlists.
- ☐ Schema validation with unknown fields rejected or safely ignored by version.
- ☐ Per-agent tool and repository authorization independent of message content.
- ☐ Explicit approval for money movement, public posting, deletion, credential changes, and production deployment.
- ☐ Outbound data-loss-prevention scan before every public or private GitHub write.
- ☐ Secret references resolved only by the intended worker and never echoed into status comments.
- ☐ Emergency stop that disables dispatch while preserving inbound events.

Retention and deletion

Set a policy by classification: public threads may be permanent; internal coordination can be retained for a defined operational window; confidential payload references should expire quickly. Do not rely on deleting an issue or rewriting Git history as a secrecy mechanism. Once a credential reaches GitHub, treat it as exposed: rotate or revoke it, then remove traces as cleanup.

Hard rule: raw passwords, API tokens, private keys, session cookies, one-time codes, recovery links, and unredacted personal or customer records never enter either repository.

Start with one route and earn complexity

Phased rollout

- 0 **Paper test.** Finalize schema, labels, classification rules, approval gates, and agent registry. Create no automation yet.
- 1 **Private pilot.** One private bus repo, SAO as sender, one low-risk worker as receiver, webhook ingress through n8n, and no encrypted payloads.
- 2 **Reliable transport.** Add durable queueing, deduplication, reconciliation polling, metrics, dead-letter handling, and emergency stop.
- 3 **Public lane.** Add outbound classification, review gates, public templates, and a strict allowlist of task types.
- 4 **Sensitive references.** Add short-lived vault/object references and per-worker authorization. Add recipient encryption only for a demonstrated need.

Pilot acceptance tests

- ☐ A valid issue reaches exactly one worker and produces one acknowledgement.
- ☐ A duplicate webhook does not duplicate execution or comments.
- ☐ An invalid signature, unknown sender, or malformed envelope executes nothing.
- ☐ An out-of-order progress event cannot overwrite a terminal state.
- ☐ A task requiring approval stops until the correct approval arrives.
- ☐ A secret-shaped test string is blocked from both public and private outbound writes.
- ☐ A missed webhook is recovered by reconciliation without double execution.

Service indicators

Delivery

Accepted, rejected, duplicate, and replayed events

Latency

Event-to-acknowledge and event-to-terminal time

Safety

Policy denials, approval holds, and DLP blocks

Ownership

SAO owns protocol and policy. n8n owns ingress plumbing and queue handoff. Each worker owns only its task adapter. Nelson retains approval authority for consequential actions. Telegram remains the fast control surface; GitHub remains the durable system of record.

The baseline is ready for a controlled pilot

Decisions adopted in this design

- **Transport:** Issues and comments over the GitHub REST API; repository webhooks trigger consumers.
- **Topology:** separate public and private bus repositories.
- **Authority:** GitHub records requests; SAO authorizes and dispatches them.
- **Sensitive data:** proprietary coordination may use the private lane; raw secrets never do.
- **Identity:** fine-grained personal access token for a constrained pilot, then a GitHub App for production.
- **Reliability:** at-least-once delivery with idempotent consumers and periodic reconciliation.

Open implementation choices

1. Repository owner and final names for the public and private lanes.
 2. Whether the first pilot routes SAO to Lupin or another low-risk worker.
 3. Which secret manager or encrypted object store resolves `payload_ref`.
 4. Retention periods for internal and confidential thread metadata.
 5. Whether logical agent events receive an additional SAO signature in v1.
-

Official references

[1] **Validating webhook deliveries.** GitHub Docs. <https://docs.github.com/en/enterprise-server@3.11/webhooks/using-webhooks/validating-webhook-deliveries>

[2] **Permissions required for fine-grained personal access tokens.** GitHub Docs. <https://docs.github.com/en/rest/authentication/permissions-required-for-fine-grained-personal-access-tokens>

[3] **Webhook events and payloads.** GitHub Docs. <https://docs.github.com/en/webhooks/webhook-events-and-payloads>

[4] **Generating an installation access token for a GitHub App.** GitHub Docs. <https://docs.github.com/en/apps/creating-github-apps/authenticating-with-a-github-app/generating-an-installation-access-token-for-a-github-app?search-overlay=open=true>

[5] **Keeping your API credentials secure.** GitHub Docs. <https://docs.github.com/en/enterprise-cloud@latest/rest/authentication/keeping-your-api-credentials-secure>

[6] **About push protection.** GitHub Docs. <https://docs.github.com/en/code-security/concepts/secret-security/about-push-protection>

[7] **Rate limits for the REST API.** GitHub Docs. <https://docs.github.com/rest/using-the-rest-api/rate-limits-for-the-rest-api>

[8] **Troubleshooting webhooks.** GitHub Docs. <https://docs.github.com/en/enterprise-cloud@latest/webhooks/testing-and-troubleshooting-webhooks/troubleshooting-webhooks>

Reference links checked September 25, 2026. This document defines the proposed architecture; it does not create repositories, credentials, webhooks, or automations.